Continuous Threat Exposure Management System

TRD

(Recommended)

Folder structure:

```
C-TEMS/
├── /frontend                ATHARVA
└── /backend
    ├── .env
    ├── .env.example
    ├── .gitignore
    ├── requirements.txt
    ├── main.py              ATHARVA
    ├── /helpers             (this must contain all the model evaluation/research codes)
    ├── /data
    │   └── google_oauth_scopes.csv
    ├── /core
    │   ├── config.py
    │   └── database.py
    ├── /models
    └── /services
        ├── /overview        SAMRUDHI
        ├── /reconciliation  PARESH
        ├── /blast_radius    SHRAVANI
        ├── /temporal_risk   SAMRUDHI
        ├── /remediation     PRASANNA
        └── /audit           ATHARVA
```

Note: the "helpers" folder must be updated by **everyone**, it must contain the model evaluation codes, data preparing codes etc. anything of somewhat relevance.

FRONTEND - ATHARVA

BACKEND:

1. OVERVIEW (The Executive View) - SAMRUDHI

1. Security Posture Dashboard

Backend Tech & ML: Python pandas, PostgreSQL

How & Why: The Python server executes an aggregation query on the PostgreSQL database to fetch the current state of all scanned scripts. Using pandas, it joins this data with the in-memory google_oauth_scopes.csv mapping.

The Math: It calculates the Risk Exposure Score (RES) by multiplying the frequency of an API call by its mapped sensitivity weight. For the Quadrant Chart, Python calculates the Jaccard Distance between requested and used scopes for the X-axis, and retrieves the Cosine Similarity score (calculated by the NLP engine) for the Y-axis.

2. Recent Anomalies

Backend Tech & ML: IsolationForest (scikit-learn)

How & Why: We use Unsupervised ML here because it requires zero labeled training data and no costly API calls. The Python backend constantly feeds execution logs (time of execution, volume of API calls, user role) from PostgreSQL into the local IsolationForest model. The algorithm isolates data points; normal behavior requires many steps to isolate, while anomalous zero-day abuse requires very few. It assigns an anomaly score (-1 to 1). Scores close to -1 trigger the backend to write an 'Alert Event' to the SQL database.

2. CODE RECONCILIATION (The Core Tool) - PARESH

1. Repository Scanner

Backend Tech & ML: Python zipfile, os, requests, Clerk Python SDK

How & Why: The server receives a .zip stream or uses requests to download a GitHub repository zipball. Python extracts this into a secure, quarantined temporary directory (tempfile module). It traverses the files, aggressively discarding anything that is not a .gs or appsscript.json file to prevent malicious server injection. The Clerk SDK validates the user's JWT token.

2. Workspace File Tree

Backend Tech & ML: Python Dictionary Mapping and Set Logic

How & Why: As Python maps the directory, it uses native Set Mathematics. It creates two sets: S_requested (from the JSON file) and S_used (from the AST parser). It executes S_requested - S_used. If the resulting set is not empty, the backend tags the file as Overprivileged. It then performs an O(1) lookup against the loaded CSV dataset to determine severity.

3. Split-Pane Inspector

Backend Tech & ML: tree-sitter-javascript (AST), sentence-transformers (all-MiniLM-L6-v2)

How & Why: Deterministic Tech: tree-sitter-javascript deterministically builds an Abstract Syntax Tree (AST) offline to map out every single CallExpression.

NLP Tech: To keep costs at absolute zero, Python's regex module extracts only human comments from the .gs file. The local MiniLM model embeds these comments into a 384-dimensional vector. It uses scipy.spatial.distance.cosine to compare this vector against pre-embedded vectors of Google Scope descriptions to prove intent.

3. BLAST RADIUS (The Graph ML Layer) - SHRAVANI

1. Shadow Automation Graph

Backend Tech & ML: NetworkX (Python Graph Theory library)

How & Why: The Flask server queries PostgreSQL for all users, scripts, and Drive files. It uses NetworkX to construct a Directed Acyclic Graph (DAG) in server memory. Users and Files are mapped as Nodes (V), and the OAuth scopes connecting them are mapped as Directed Edges (E).

2. Exposure Simulator

Backend Tech & ML: Weighted Breadth-First Search (BFS) algorithm

How & Why: When a breach simulation is triggered, the Python backend executes a BFS traversal through the NetworkX graph starting from the compromised script node. It calculates the Predictive Blast Radius by applying a decay formula: it sums the sensitivity weights of all reachable downstream files, divided by their hop-distance from the compromised node.

4. TEMPORAL RISK (The Time-Based Layer) - SAMRUDHI

1. JIT (Just-In-Time) Policies

Backend Tech & ML: Python APScheduler / Celery, PostgreSQL

How & Why: The Flask API writes JIT rule parameters into a PostgreSQL table. A background Python worker running on an endless loop periodically executes a SQL INNER JOIN between the jit_policies table and the execution_logs table. If current_date - last_used_date > policy_limit, it flags the token for automated revocation.

2. Scope Decay Analytics

Backend Tech & ML: Python numpy, PostgreSQL window functions

How & Why: The Python backend queries the execution logs to find the last active timestamp of a permission. It uses numpy to apply an exponential decay function (y = e^(-kt), where t is time elapsed). This generates a mathematical curve representing how 'stale' the permission is.

5. REMEDIATION CENTER (Developer Action) - PRASANNA

1. Pull Request Manager

Backend Tech & ML: Pure Python JSON manipulation, PyGithub

How & Why: To avoid LLM hallucinations, this step is 100% deterministic. Python opens the user's appsscript.json file in memory, deletes the overprivileged oauthScopes array, and replaces it with the exact S_min array calculated by the AST Parser. It then uses the PyGithub library to authenticate, create a branch, and open a PR automatically. No generative AI is used to write code.

2. Friction Analytics

Backend Tech & ML: Flask Webhook routing, PostgreSQL

How & Why: The backend registers a webhook with GitHub. When a developer clicks 'Merge' or 'Close' on the PR, GitHub fires a JSON payload to the Python server. Python updates the pr_status column in the database and calculates the Remediation Acceptance Rate.

6. AUDIT & COMPLIANCE (The Security Log) - ATHARVA

1. Scope change Ledger

Backend Tech & ML: PostgreSQL, SQLAlchemy ORM

How & Why: We abandon complex blockchain hashing in favor of strict application-layer governance. The ledger's immutability is guaranteed because the Python Flask router simply does not contain DELETE or PUT endpoints for the audit_logs table. The API only accepts POST requests to append new log rows.

2. Point-in-Time Forensics

Backend Tech & ML: PostgreSQL (Event Sourcing Pattern)

How & Why: The database strictly logs state changes (deltas) with a created_at timestamp. When an auditor queries a historical date, the Python backend runs: SELECT * FROM audit_logs WHERE created_at <= [DATE]. It sequentially replays these database events in server memory to perfectly reconstruct the exact graph topology and permission matrix that existed at that precise millisecond.

7. Main.py (containing all restful api endpoints) - ATHARVA

Note

1. Whilst working on the assigned feature, also do note your Mathematical and Tech approaches, as it will prove fruitful in paper formation.
2. Create your assigned files, I'll add 'em in the main.py later (as functions are to be mentioned).